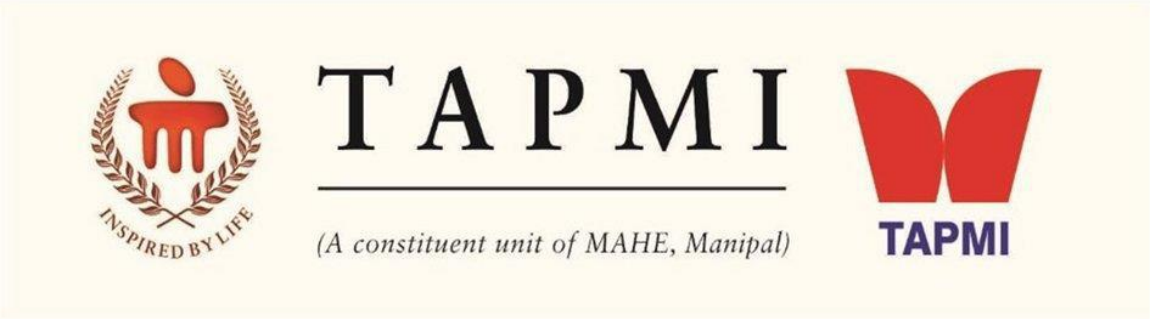
CYBERSECURITY PROJECT REPORT

ARCHITECTURE



NAME: POOJITHA K

ID: 251620140002

The technology stack (all open source)

Everything is free and open source. The table lists each layer.

Layer	Choice	Why it was chosen
Language	Python 3	Readable has strong security tooling
Web framework	Flask (BSD licence)	Small and easy to read and audit
Templating	Jinja2	Escapes output automatically, which stops most XSS
Database	SQLite	Zero setup; holds users, accounts, transactions, payees
Password storage	Werkzeug (scrypt)	One-way hashing - passwords can't be reversed
Authentication	Password + simulated SMS OTP	Two factors, like a real bank (RBI-mandated OTP)
CSRF protection	Flask-WTF	Adds a secret token to every form that changes data
Static scanner (SAST)	Bandit	Reads the code and flags insecure patterns
Dependency	pip-audit	Checks libraries for known

scanner (SCA)		vulnerabilities
Reverse proxy (production)	Nginx	Handles HTTPS/TLS and a first layer of filtering

How a request flows, and where security sits

The portal has a banking layout with a sidebar: Dashboard, Accounts, Transfer, Transactions, Payees, Cards, Support and Profile, plus the login and OTP pages. The diagram shows the path a request takes and the security control at each layer.

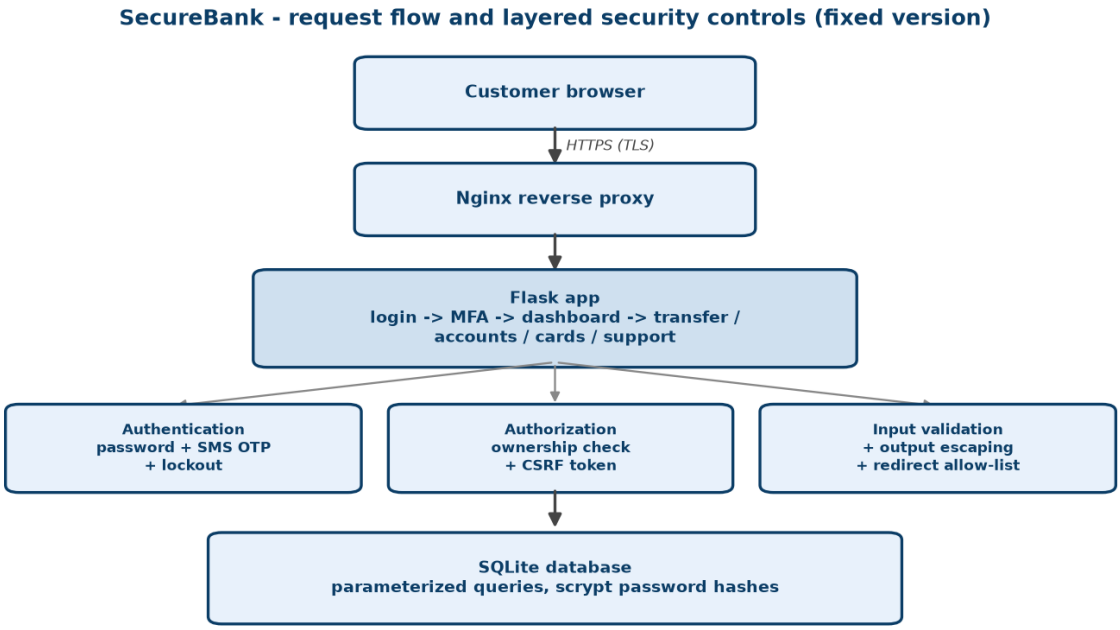


Figure 1. Request flow and layered security controls (fixed version).

Key design decisions

- **Least privilege -** every query is limited to the logged-in user; opening another person's account is refused.
- **Defense in depth -** the browser limits input, the server checks it again, output is escaped, database is reached only through safe parameterized queries.
- **Secrets in the environment -** the session key is read from an environment variable; the app refuses to start without it.
- **Two-factor by default -** a password alone is never enough - an SMS OTP is always required.

Each layer adds its own defense, so the portal is secure by design.
The open-source tooling lets anyone verify every choice independently.